

```
//Mainwindow.h
#ifndef MAINWINDOW_H
#define MAINWINDOW_H
#include <QTimer>
#include <QVector>
#include <QPoint>
#include "pacman.h"
class MainWindow : public QMainWindow {
    Q_OBJECT
public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();
protected:
    void keyPressEvent(QKeyEvent *event) override; // Handles keyboard input
    void paintEvent(QPaintEvent *event) override; // Responsible for drawing the game screen
private slots:
    void gameLoop(); // Runs every tick to update game logic
private:
    Pacman *pacman; // Pacman character instance
    QTimer *timer; // Timer to control game updates
    int score; // Player score
    QVector<QPoint> pellets; // Vector holding pellet positions
    void initializePellets(); // Setup initial pellet layout
    // Enemy logic
    QPoint enemyPos; // Current position of the enemy
    int enemyDx, enemyDy; // Direction of enemy movement
    void moveEnemy(); // Update enemy position
};
#endif // MAINWINDOW_H
-----
//pacman.h
#ifndef PACMAN_H
#define PACMAN_H
#include <QWidget>
#include <QPainter>
class Pacman : public QWidget {
    Q_OBJECT
public:
    explicit Pacman(QWidget *parent = nullptr);
    void move(); // Update Pacman's position
    void draw(QPainter *painter); // Draw Pacman
    void setDirection(int key); // Change movement direction
```

By Rukaia Abdelhameed, Rawan, Mariam,Nourhan Ghonaem

```
// FILL THE BELLOW SETTER AND GETTER TO RETURN THE X AND Y POSITION
int getX() { return x; }
int getY() { return y; }
private:
    int x, y; // Pacman's position
    int dx, dy; // Movement vector
};
#endif // PACMAN_H
```

```
-----
//main.cpp
#include "mainwindow.h"
#include <QApplication>

int main(int argc, char *argv[]) {
    QApplication app(argc, argv);
    MainWindow window;
    window.show();
    return app.exec();
}
```

```
-----
//mainwindow.cpp
#include "mainwindow.h"
#include "pacman.h"
#include <QPainter>
#include <QTimer>
#include <QKeyEvent>
#include <QPixmap>
#include <QDebug>
#include <cstdlib>
#include <ctime> //our attempt to use time
```

```
MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent), pacman(new Pacman(this)), timer(new QTimer(this)), score(0),
    enemyPos(100, 100), enemyDx(10), enemyDy(0)
{
    setFixedSize(400, 400); /// Set the window size
    std::srand(std::time(nullptr)); //
    initializePellets(); //creating pellets at fixed position
    connect(timer, &QTimer::timeout, this, &MainWindow::gameLoop);
    timer->start(100); //Game updates every 100ms
```

By Rukaia Abdelhameed, Rawan, Mariam,Nourhan Ghonaem

```
}
```

```
MainWindow::~MainWindow() {}
```

```
void MainWindow::keyPressEvent(QKeyEvent *event) {  
    pacman->setDirection(event->key());  
}
```

```
//move pacman based on arrow key
```

```
void MainWindow::initializePellets() {  
    for (int y = 20; y < 400; y += 20) {  
        for (int x = 20; x < 400; x += 20) {  
            pellets.append(QPoint(x, y));  
        }  
    }  
}
```

```
//to place pellets in a grid pattern, adding space in the window for the pellets while not hitting  
edge of window, leaving 20 units before the screen ends
```

```
void MainWindow::moveEnemy() {  
    enemyPos.rx() += enemyDx;//move enemy  
    enemyPos.ry() += enemyDy;//move enemy  
  
    if (enemyPos.x() <= 0 || enemyPos.x() >= width() - 30) {  
        enemyDx = -enemyDx;  
    }  
    if (enemyPos.y() <= 0 || enemyPos.y() >= height() - 30) {  
        enemyDy = -enemyDy;  
    }  
}
```

```
//bouncing off window edges
```

```
void MainWindow::gameLoop() {  
    pacman->move();//move pacman  
    moveEnemy();//move enemy
```

```
    QRect pacmanRect(pacman->getX(), pacman->getY(), 30, 30);
```

```
    for (int i = 0; i < pellets.size(); ++i) {  
        QRect pelletRect(pellets[i].x(), pellets[i].y(), 10, 10);  
        if (pacmanRect.intersects(pelletRect)) {  
            pellets.remove(i);//remove pellet on collision  
            score++;//increase score  
            break;//only eat one pellet at a time  
        }  
    }  
}
```

By Rukaia Abdelhameed, Rawan, Mariam,Nourhan Ghonaem

```
    QRect enemyRect(enemyPos.x(), enemyPos.y(), 30, 30);
    if (pacmanRect.intersects(enemyRect)) {
        timer->stop();//stop game on collision with enemy
        // Game Over logic here
    }

    update();
}

void MainWindow::paintEvent(QPaintEvent *) {
    QPainter painter(this);

    // Draw Pacman
    pacman->draw(&painter);

    // Draw pellets
    QPixmap pelletImg(":/images/pics/pellet 2.png"); // Use resource path
    for (const QPoint &p : pellets) {
        painter.drawPixmap(p.x(), p.y(), 10, 10, pelletImg);
    }

    // Draw enemy
    QPixmap enemyImg(":/images/pics/ghost villian.PNG");
    painter.drawPixmap(enemyPos.x(), enemyPos.y(), 30, 30, enemyImg);

    // Draw score
    painter.setPen(Qt::black);
    painter.setFont(QFont("Arial", 14));
    painter.drawText(10, 20, QString("Score: %1").arg(score));
}
```

```
//pacman.cpp
#include "pacman.h"
#include <QKeyEvent>
#include <QPixmap>

// Constructor to position the pacman in its initial position
Pacman::Pacman(QWidget *parent)
    : QWidget(parent), x(200), y(200), dx(0), dy(0) {}

// Set direction based on key press
```

```
void Pacman::setDirection(int key) {
    switch (key) {
        case Qt::Key_Up:
            dx = 0;
            dy = -10; // Move up (negative y direction)
            break;
        case Qt::Key_Down:
            dx = 0;
            dy = 10; // Move down (positive y direction)
            break;
        case Qt::Key_Left:
            dx = -10;
            dy = 0; // Move left (negative x direction)
            break;
        case Qt::Key_Right:
            dx = 10;
            dy = 0; // Move right (positive x direction)
            break;
        default:
            break;
    }
}

// Update Pacman's position
void Pacman::move() {
    x += dx; // dx is horizontal speed
    y += dy; // dy is vertical speed

    // Ensure Pacman stays within window bounds (assuming 400x400 window, and Pacman size
    // is 30)
    if (x < 0) x = 0;
    if (x > 370) x = 370;
    if (y < 0) y = 0;
    if (y > 370) y = 370;
}

// Draw Pacman using an image
void Pacman::draw(QPainter *painter) {
    QPixmap pacmanImg(":/images/pics/pacman.png");
    painter->drawPixmap(x, y, 30, 30, pacmanImg);
}
```